Reproducing FACIL Address Mapping and Modeling Batch-Aware Decode Dispatch for SoC–PIM LLM Inference

Sukhjot Singh, T S S Sreeja, Mayank Singh, and Pankaj
Group 9, Department of Computer Science and Engineering
Indian Institute of Technology Guwahati

***Abstract*—FACIL uses page-selected DRAM address mappings to share one weight layout between a system-on-chip (SoC) and processing-in-memory (PIM) units. We reproduce this mechanism for four Llama3-8B FP16 linear projection classes with a reference mapper and a modified DRAMsim3 decoder and channel router. Across 32 decoder blocks, the sum of independently simulated source READ and destination WRITE completion brackets is 447.284 ms. In a matched two-region K/V test, a preplaced conventional copy reduces the SoC read scan from 184.505 to 41.412 μs while preserving MapID 2 placement for PIM. Duplicating all 224 modeled linear matrices increases their calculated weight capacity from 13 to 26 GiB. We use the traffic results to construct projection-only prefill scenarios and extend FACIL with a batch-aware decode policy. At the selected SoC efficiency, the first modeled SoC-favorable batch is 17 or 5 under two hypothetical PIM timing scenarios. An offline scheduling analysis quantifies how much uniform switching overhead the policy comparison can tolerate. The measurements cover simulated DRAM traffic. First-token latency, compute performance, and engine transitions are treated analytically.**

***Index Terms*—DRAM address mapping, processing-in-memory, LPDDR5, DRAMsim3, LLM inference, batch-aware decode, execution dispatch, reproducibility.**

I. Introduction

On-device LLM inference uses the same weight matrices in distinct computational regimes. During prefill, many input tokens are processed with matrix–matrix multiplication (GEMM), favoring the SoC's GPU/NPU. During autoregressive decode, matrix–vector multiplication (GEMV) is memory-intensive and can benefit from bank-level PIM [1]. Near-bank PIM requires a specific distribution of weight chunks over DRAM banks, whereas standard SoC software expects logically row-major (or column-major) matrices. The on-demand re-layout baseline copies and rearranges weights before SoC GEMM; FACIL instead makes each huge page's physical-to-DRAM address mapping selectable while preserving conventional virtual addressing [1].

Reproduction objective. The reduced-scope assignment narrows the original paper to a standalone mapping model, placement validation, DRAMsim3 re-layout-cost comparisons, and analytical PIM GEMV/TTFT estimates. The study does not reproduce modified OS page tables, actual SoC measurements, or a cycle-accurate PIM compute engine. The comparison is the *baseline with on-demand re-layout* versus the *same SoC–PIM architecture with FACIL-style mapping and no extra re-layout*. An additional implemented policy is evaluated separately.

Questions. (i) Can the paper's MapID selection and chunk placement be reproduced? (ii) What DRAM traffic cost is associated with the baseline's extra copy? (iii) What does eliminating that cost imply for analytical TTFT, subject to explicit SoC/PIM model assumptions? (iv) When can batch-width-aware dispatch improve modeled decode linear-layer latency with a single weight copy?

II. FACIL Mechanism and Comparison Boundary

A. Mapping and tensor placement

FACIL associates a mapping identifier (MapID) with a huge page. The MapID selects which page-offset bits are interpreted as DRAM row, bank, rank, channel, column and transfer-offset bits. In the AiM-style mapping shown in Figs. 8–10 of [1], the low bits keep each $(1,1024)$-element FP16 PIM chunk contiguous within a 2 KiB DRAM row. The PU-changing bits are placed above a MapID-controlled number of DRAM-row bits. Multiple mapping choices allow different matrix shapes to preserve PIM-operable placement.

Let n be the number of FP16 columns, $b = 2$ B/element, $K = 1024$ elements/chunk, $H = 2$ MiB the huge-page size, and $U = N_{\text{ch}} N_{\text{rank}} N_{\text{bank}}$ the PIM-unit count. The Fig. 9 selector is implemented using consistent byte units:

$$R_{\text{rounded}} = 2^{\lceil \log_2 n \rceil} b, \tag{1}$$

$$B_{\text{perPU}} = H/U, \tag{2}$$

$$\text{MapID} = \log_2\left(\frac{\min(R_{\text{rounded}}, B_{\text{perPU}})}{Kb}\right), \tag{3}$$

$$\text{partition} = \left[R_{\text{rounded}} > B_{\text{perPU}}\right].$$

This byte-normalized expression implements the intended bit-count shown by the paper's examples; the printed pseudocode mixes column-count and byte-size nomenclature. The independent mapping is reversible and is also integrated in the native DRAMsim3 decoder and channel router for this reduced-scope test.

B. Fair comparison boundary

The on-demand baseline retains a PIM-operable weight representation for decode and converts the matrix when the SoC needs its conventional layout for prefill GEMM. The

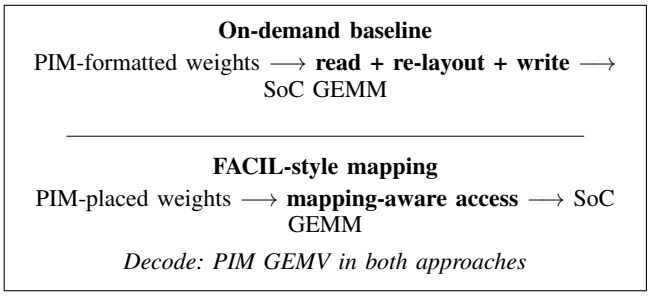


Fig. 1. Comparison boundary. The absence of an extra *re-layout* operation is architectural; it is not a second DRAMsim3 timing measurement. Computation and possible mapping-induced GEMM penalties must be accounted for separately.

destination is a separate DRAM region. GPU caches, registers, and tile buffers serve a different role during computation. FACIL removes that conversion by allowing the SoC to address the PIM-optimized physical placement through normal virtual addresses. The original paper's Fig. 5 motivates this comparison [1].

III. Implementation and Experimental Setup

A. Geometry and simulator

The simulator uses four DRAMsim3 channels, two ranks/channel, 16 banks/rank, 2-MiB huge pages, and a 2 KiB AiM reference chunk consisting of 1,024 FP16 elements. The reference map decomposes a PIM chunk into 32-B DRAM transfers. Stock DRAMsim3 traces in this study instead issue 128-B transactions; each transaction groups four contiguous 32-B reference transfers; the trace generator verifies that all four map to the same channel/rank/bank/DRAM row with consecutive columns.

The `LPDDR5_6400_APPROX_v1.ini` configuration uses `rows=262144`, `columns=256` per device, four 64-bit channels, two ranks/channel, 16 banks/rank, and `BL=16`. The resulting *aggregate* bank-row capacity is $256 \times 64/8 = 2\,\text{KiB}$. This is a synthetic row geometry chosen for the AiM chunk model, not a validated physical LPDDR5 organization.

DRAMsim3 in this checkout implements an LPDDR4 command protocol, with selected LPDDR5-6400-like timing parameters and $t_{CK} = 0.3125\,\text{ns}$. The ideal 256-bit aggregate bandwidth at 6400 MT/s is 204.8 GB/s, *not* an observed read throughput and not proof of native LPDDR5 fidelity. The paper used a separately modified DRAMsim-based PIM/LPDDR5 simulator [1]. Baseline re-layout source traces use verified physical-address encodings for a conventional decoder. The *native MapID-aware decoder and channel router* were implemented and validated in the DRAMsim3 checkout, with a static per-huge-page registry supplied through `FACIL_PAGE_MAP_FILE`. This is software modeling, not actual PTE-borne MapIDs, OS modification, or a PIM MAC engine.

Trace-reader integrity. A one-WRITE EOF regression test produces exactly one WRITE and zero READs. Baseline stage counts are checked against trace manifests and DRAMsim3 transaction statistics.

B. Full-model linear-weight workload accounting

The Llama3-8B configuration contains 32 decoder blocks; each block uses two Q/O projections, two K/V projections, two FFN gate/up matrices, and one FFN down matrix. The four distinct matrix shapes listed in Table I therefore account for seven linear weight matrices per block, totaling 416 MiB of FP16 weights per block. Across 32 blocks, the enumerated linear weights occupy 13.0 GiB, excluding embeddings, the output head, normalization tensors and the attention cache. This is computed from real model dimensions, not inferred from the small proxy runs.

A once-per-matrix read-plus-write traffic-only conversion would move 26.0 GiB for these linear weights. Even at the paper's Jetson reference *peak* external bandwidth of 204.8 GB/s, this requires at least 136.3 ms of bus service if all these bytes cross that interface, excluding all other work. This value is an *optimistic bandwidth floor*, not a measured baseline re-layout latency and not TTFT. The aggregate may change if weights are reused, transfers overlap, or actual software routes differ. The included workload-accounting script calculates the matrix sizes and transfer volume from tensor dimensions.

C. Address-placement validation

For every 1024-element FP16 chunk in four Llama3-8B matrix shapes, DRAM-row and PU containment, boundary alignment, and encode–decode round trips were verified. Row-wise PU counts and crossings of 2-MiB page boundaries were also checked. Python unit tests cover Figure 10 anchors, mapping invertibility and selected per-page-ID behavior. A sampled numerical GEMV check computes chunk-local partial sums and compares their reduction against ordinary dot products. These are *functional and placement* tests, not bank lock-step execution tests.

D. Native MapID decoder and channel routing prototype

As an additional correctness gate, the existing DRAMsim3 C++ `Config::AddressMapping()` was extended to optionally choose an AiM-style bit mapping from a static MapID registry for each 2-MiB huge page. Crucially, `BaseDRAMSystem::GetChannel()` was also changed to use the same decoded channel; otherwise requests could be routed to a conventional channel while the command scheduler decoded a different one. Pages without registered MapIDs preserve conventional decoding. The registry represents a reduced-scope test fixture, *not* modified OS page tables or a native PIM controller.

Independent Python/C++ validation checked 996 random/boundary addresses and 2,162,688 non-aliasing 128-B blocks. A further 411 independent native decoder comparisons agreed, both MapID 2 and MapID 3 passed exhaustive single-page no-alias tests, and 16,384 accesses on an unregistered page preserved conventional decoding. A 32-request routing test showed conventional per-channel counts $(8,8,8,8)$ versus MapID 2 counts $(32,0,0,0)$. A 96-request mixed-page test produced $(72,8,8,8)$ with zero writes. These tests establish

TABLE I
TARGET LLAMA3-8B FP16 MATRICES (GEOMETRY FROM THE ORIGINAL PAPER'S LAYER DIMENSIONS).

Layer class	Rows	Cols	Size	MapID
Q/O	4,096	4,096	32 MiB	2
K/V	1,024	4,096	8 MiB	2
FFN gate/up	14,336	4,096	112 MiB	2
FFN down	4,096	14,336	112 MiB	3

decoder and routing correctness, *not* GEMM/PIM compute execution.

E. Functional baseline re-layout verification

Weight integrity was verified independently of the DRAMsim3 timing engine. In the standalone software storage model, the PIM-oriented physical representation is read through both an SoC logical-address view and a PIM bank/row view; the on-demand baseline then reconstructs the same logical weights into conventional contiguous storage. Bytewise tests covered the four shapes in Table I, a total of 2,162,688 distinct 128-B blocks; all compared equal. Additional sample FP16 dot-product checks for K/V and FFN down passed. These tests establish *software-model data correctness*, not actual memory-controller MapID support or PIM MAC execution.

F. Baseline re-layout traffic experiment

For the 2 KiB-row configuration, each logical 128-B weight block is read from its statically encoded PIM-oriented source and written to a disjoint conventional row-major destination. Every four underlying 32-B fragments, source uniqueness, destination alignment, and channel counts are checked; for the K/V case this means 262,144 32-B fragments and 65,536 distinct 128-B requests per stage. DRAMsim3 replays *transactions*, not the actual CPU-side byte permutation.

The READ and WRITE stages run independently with the same DRAMsim3 configuration and 256-cycle completion-search resolution. The reported sum

$$\widehat{T}_{\text{re-layout}} = \widehat{T}_{\text{source read}} + \widehat{T}_{\text{destination write}} \tag{4}$$

is a *serialized sum of DRAM completion-time upper brackets*. It is *not* a rigorous bound on actual CPU/GPU conversion time: CPU data rearrangement, caches, staged buffering and read/write overlap are outside the simulator. The original paper similarly excluded in-page reordering overhead from its DRAM-only re-layout estimate [1]. The absence of the extra copy under FACIL is a mechanism, not a second measured copy trace.

IV. BASELINE RE-LAYOUT EVALUATION

A. Baseline re-layout traffic and completion times

Four shape-specific traces were evaluated with `measure_relayout.py` and the 2-KiB bank-row configuration. Every source READ and destination WRITE stage completed the expected request count: 65,536 for K/V, 262,144 for Q/O, and 917,504 for each FFN matrix. Each

TABLE II
BASELINE DRAMSIM3 SERIALIZED STAGE-COMPLETION UPPER BRACKETS UNDER THE 2-KIB AGGREGATE BANK-ROW CONFIGURATION. ALL VALUES ARE IN μS. THESE ARE READ/WRITE TRAFFIC PROXIES, NOT CPU REORDERING OR GPU COMPUTE TIMES.

Matrix	Weight (MiB)	Read (μs)	Write (μs)	Serial sum (μs)
Q/O	32	736.000	165.920	**901.920**
K/V	8	184.560	41.120	**225.680**
FFN gate/up	112	2575.300	581.910	**3157.210**
FFN down	112	4826.080	581.910	**5407.990**

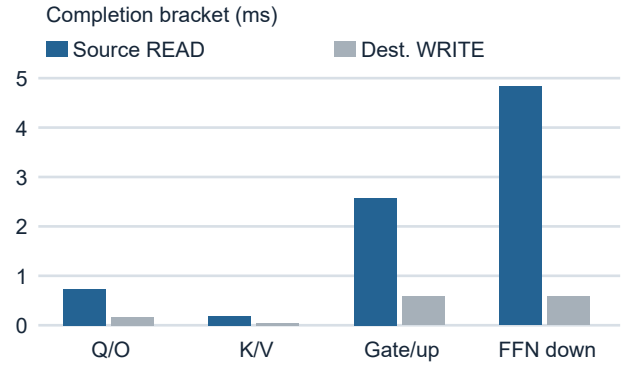


Fig. 2. Source READ and destination WRITE completion brackets for the serialized DRAM re-layout proxy. Table II reports their sums.

stage's completion is reported using an upper bracket with 256-cycle resolution (0.080 μs at $t_{CK} = 0.3125$ ns).

The FFN-down source READ bracket is 4.826 ms, compared with 2.575 ms for the equal-sized gate/up matrices (Fig. 2). A compact FFN-down row contains 14 chunks, whereas one PU receives eight chunks per huge page in the modeled geometry. The row therefore spans PUs. Placement and trace order both affect memory service, and this comparison does not isolate their individual contributions. The 204.8-GB/s Jetson reference is a theoretical aggregate peak.

For 32 decoder blocks and per-block matrix multiplicities (Q/O:2, K/V:2, gate/up:2, down:1), the *sum* of these independently measured stage upper brackets is

$$\begin{aligned}\widehat{T}_{\text{copy},32} &= 32\big[2(0.901920) + 2(0.225680) \\ &\quad + 2(3.157210) + 5.407990\big] \\ &= 447.28352\,\text{ms}.\end{aligned} \tag{5}$$

This assumes every modeled weight matrix is converted once, all reads/writes execute serially, and all stages retain the same isolated DRAM conditions. It is *not* a measured total LLM re-layout, a bound on host-side byte permutation, or a measured TTFT. The timing model uses trace manifests, independent READ/WRITE transaction statistics, and the same configuration for each matrix class. Only the DRAM traffic component is included in the reported copy-stage sum.

TABLE III
MATCHED SEQUENTIAL READ COMPLETION UPPER BRACKETS IN DRAMSIM3 (μS); NO EXTRA COPY IN FACIL.

Weight matrix	Reads	Conventional	FACIL MapID
Q/O	262,144	166.389	736.000
K/V	65,536	41.443	184.509
FFN gate/up	917,504	582.301	2575.300
FFN down	917,504	582.301	4826.080

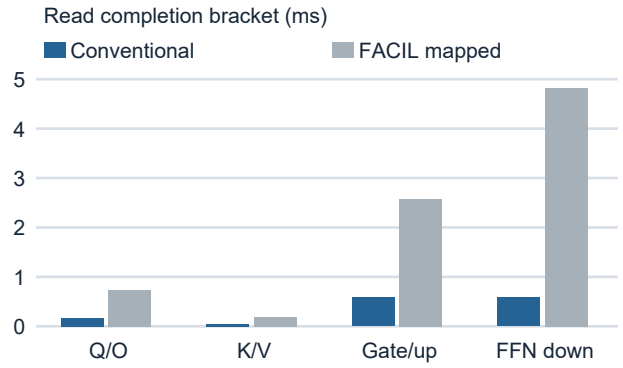


Fig. 3. Conventional and native FACIL sequential read completion upper brackets. The on-demand source READ and destination WRITE stages are shown separately in Fig. 2.

TABLE IV
MATCHED NATIVE DRAMSIM3 K/V SCAN AND CALCULATED WEIGHT CAPACITY

Metric	Shared FACIL	Two copies
Completed reads	65,536	65,536
Completion cycles	590,416	132,517
SoC scan (μs)	184.505	41.412
Effective bandwidth (GB/s)	45.465	202.567
K/V logical weight footprint	8 MiB	16 MiB
224-matrix calculated footprint	13 GiB	26 GiB
PIM placement	MapID 2 pass	MapID 2 pass

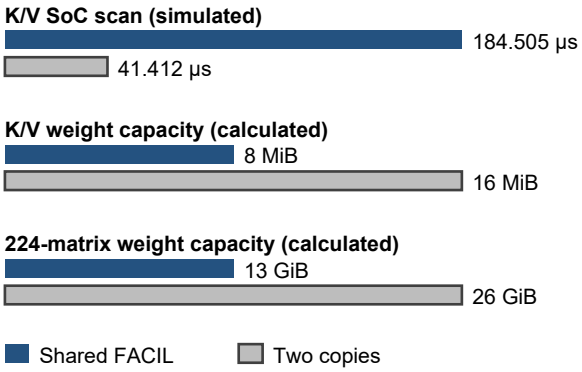


Fig. 4. Simulated read completion for one K/V matrix and calculated FP16 capacity across all 224 linear matrices. The bars use different scopes. Each bar pair uses its own scale.

V. FACIL MAPPING AND MEMORY-ACCESS EVALUATION

A. FACIL MapID and chunk placement

Under Fig. 10's two-channel example geometry, the reproduced selector returns MapID 3 for 2048×8192 and MapID 5 for 2048×65536, as in [1]. Under the four-channel, two-rank geometry, Table I gives the respective values for the four actual Llama3-8B shapes. Across these shapes, 135,168 chunks pass the implemented placement checks; 13 Python unit tests pass. The first three matrix shapes place each logical row within one PU. For the 4096×14336 compact FFN-down shape, 2,048 rows use two PUs, 2,048 use three PUs, and 48 rows cross huge-page boundaries. These findings reproduce the selector and demonstrate reversible placement; they do *not* establish PIM lock-step timing.

B. Native FACIL matched-read comparison

The same sequence of useful FP16 weights was traced through the conventional post-re-layout SoC view and the original PIM-placed page(s) under native MapID decoding. All four categories pass source/destination address-identity tests, all expected requests complete, and no writes are emitted. The mapping differences in Table III are *DRAM read* differences, not GPU GEMM slowdowns. The completion searches report upper brackets at 0.3125 ns DRAM cycles. FACIL scans take 4.423–8.288 times as long as conventional scans for these traces, with the highest ratio for FFN down. The projection model treats the read difference as a sensitivity input.

C. Preplaced two-copy comparison

The matched-read study above compares conventional and MapID decoding across four matrix classes. To check that both layouts can coexist, we also register a MapID 2 region and a disjoint conventional region for one 1024×4096 FP16 K/V matrix. The native verifier checks conventional fallback, agreement with the reference mapper, absence of address aliases, and PU locality. A mixed trace checks channel completion counts for the same registry. All ten recorded mapper tests passed.

Both SoC runs replay 65,536 reads of 128 bytes (8,388,608 bytes) at identical arrival cycles. We search for the first cycle budget at which all requests complete and confirm that one cycle less is insufficient. The traces match the historical K/V traces byte for byte. The finer search places completion 13 cycles below the earlier FACIL upper bracket and 99 cycles below the conventional bracket.

The conventional copy reduces the simulated K/V scan time by 143.093 μs (77.6%) and gives 4.455$\times$ the effective bandwidth (Table IV and Fig. 4). This comparison assumes the layouts are prepared in advance and measures sequential DRAM reads. The 224-matrix row accounts for model weights by shape. Matched two-region timing covers K/V. The other classes retain their conventional-versus-mapped read comparisons. The ideal decode comparator in the next part of the paper uses equal engine costs for the two storage strategies.

TABLE V
ANALYTICAL SOC PREFILL *linear projection* GEMM PEAK-ROOFLINE ESTIMATES ON JETSON (NOT MEASURED GPU TIME, NOT FULL TTFT).

P	GEMMs	Wt. (GiB)	Time (ms)
8	224	13.0	68.362
16	224	13.0	68.567
32	224	13.0	68.977
64	224	13.0	69.796
128	224	13.0	71.434

VI. ANALYTICAL FIRST-TOKEN LATENCY MODELS

A. SoC peak-roofline projection model

For each FP16 linear projection with weights $W \in \mathbb{R}^{M\times K}$ and P prefill tokens, the model uses $X \in \mathbb{R}^{K\times P}$ and $Y = WX \in \mathbb{R}^{M\times P}$. Each GEMM requires $F = 2MKP$ floating-point operations under a two-FLOP fused multiply-add convention. The modeled external traffic is $B = 2MK + 2KP + 2MP$ bytes: one FP16 read of all weights, one input read, and one output write per independent GEMM. Using FACIL Table II's Jetson AGX Orin FP16 peak compute (42.5 TFLOP/s) and theoretical bandwidth (204.8 GB/s),

$$\begin{aligned} T_{\text{GEMM}}^{\text{ideal}} &= \max(T_{\text{calc}}, T_{\text{mem}}), \\ T_{\text{calc}} &= \frac{2MKP}{42.5\times 10^{12}}, \\ T_{\text{mem}} &= \frac{2MK + 2KP + 2MP}{204.8\times 10^{9}}. \end{aligned} \tag{6}$$

This ideal estimate is summed over all 224 independent linear GEMMs in 32 decoder blocks; every projection is memory-bound at the selected $P \le 128$ under these assumptions. The accompanying dependency-free Python model self-checks FLOPs, transfer bytes, tensor dimensions, 13-GiB weights, 224 linear GEMMs, and monotonicity across prefill lengths. It excludes attention-score/value GEMMs, softmax, layer normalization, nonlinearities, embeddings, output head, cache effects, kernel launches, and resource contention. The peak-bandwidth assumption is *not* a measured DRAMsim3 bandwidth, and this is neither a calibrated GPU model nor measured end-to-end prefill. A GPU mapping slowdown cannot be established from this model alone.

B. Baseline TTFT component proxy

The baseline's *projection-only component estimate* uses (i) the 224 SoC FP16 linear-projection GEMMs modeled by the Jetson ideal peak roofline in Table V and (ii) the 447.283520-ms sum of DRAMsim3 serialized re-layout stage-completion upper brackets in Eq. (5):

$$\widehat{T}_{\text{base,TTFT}}^{\text{scope}}(P) = T_{\text{SoC,linear}}^{\text{ideal}}(P) + \widehat{T}_{\text{copy,32}}. \tag{7}$$

The table is calculated from full-precision component values before rounding. This projection-only estimate is not the actual first-token latency: attention-score/value operations, softmax, norms, nonlinearities, embeddings/output head, sampling, GPU kernel/cache effects, resource contention and possible overlap are excluded; host-side byte permutation is also excluded as in the paper's DRAM-only re-layout proxy. Furthermore, the SoC roofline assumes ideal peak throughput, while the DRAM component is a sum of *upper* completion brackets. Therefore their sum is neither a rigorous upper/lower TTFT bound nor an end-to-end TTFT measurement. The same explicitly truncated prefill boundary is applied to the FACIL comparison, without pretending that omitted work is zero in an actual LLM.

TABLE VI
REDUCED-SCOPE *baseline TTFT component proxy* FOR LLAMA3-8B (MS). THIS IS NOT FULL OR MEASURED TTFT.

Prefill P	Ideal SoC GEMM	Copy stages	Proxy
8	68.362	447.284	**515.646**
16	68.567	447.284	**515.851**
32	68.977	447.284	**516.260**
64	69.796	447.284	**517.079**
128	71.434	447.284	**518.718**

TABLE VII
REDUCED-SCOPE FIRST-TOKEN *component proxies*, MS. NONE IS MEASURED END-TO-END TTFT.

P	Baseline	Ideal	Sensitivity	Speedup
8	515.646	68.362	377.327	1.367×
16	515.851	68.567	377.531	1.366×
32	516.260	68.977	377.941	1.366×
64	517.079	69.796	378.760	1.365×
128	518.718	71.434	380.399	1.364×

C. Reduced-scope FACIL TTFT scenarios

Let $G(P)$ be the same ideal 224-projection SoC GEMM model used in Table VI, and let $C = 447.28352$ ms denote serialized baseline re-layout stage brackets. Then

$$\widehat{T}_{\text{base}}(P) = G(P) + C, \tag{8}$$

$$\widehat{T}_{\text{FACIL,ideal}}(P) = G(P), \tag{9}$$

$$\widehat{T}_{\text{FACIL,sens}}(P) = G(P) + D, \tag{10}$$

where the explicit *mapping sensitivity* is

$$\begin{aligned} D &= 32\sum_i n_i (T_{\text{FACIL,read,i}} - T_{\text{conv,read,i}}) \\ &= 308.96428 \text{ ms}. \end{aligned} \tag{11}$$

The multiplicities $n_i = (2,2,2,1)$ correspond to Q/O, K/V, FFN gate/up, and FFN down. D is obtained from whole-matrix *sequential-read* differences and hypothesizes one pass of mapping-induced overhead per projection. Because the ideal GEMM roofline already charges for weight traffic, only the *increment* is considered; even that transfer to GPU GEMM latency is *not experimentally established*. Thus the adjusted scenario is a sensitivity analysis, *not* a demonstrated FACIL GEMM slowdown.

D. Analytical AiM-style GEMV and optional short prefill

The assignment additionally requests an analytical PIM matrix–vector model. For a weight matrix divided into 2-KiB FP16 chunks, define W_i as the maximum number of chunks assigned to one processing bank under validated MapID

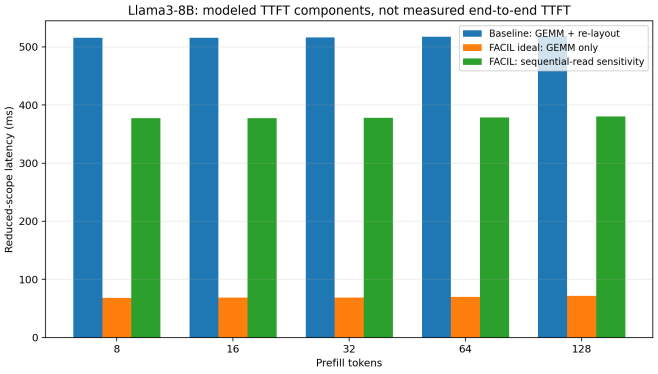


Fig. 5. Baseline, ideal FACIL, and penalty-sensitive FACIL first-token *projection-only proxy*; common prefill operations and actual GPU slowdown are not measured.

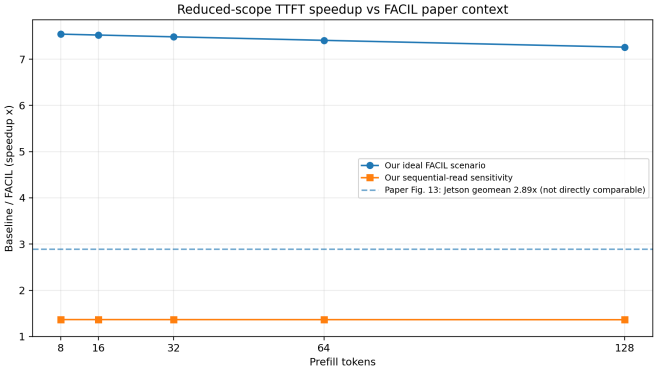


Fig. 6. Model speedups compared with the published Jetson context (2.89× geometric mean in [1]). The methodologies and workload boundaries differ; the horizontal paper reference is not a target that this proxy should numerically reproduce.

placement. The resulting counts are $W_i = 128, 32, 448, 448$ for Q/O, K/V, FFN gate/up, and FFN down. Under a *hypothetical* effective PIM processing-wave latency $\tau \in \{75, 100, 200\}$ ns, the optimistic GEMV formula is

$$\widehat{T}_{\mathrm{PIM,GEMV,i}} = W_i \tau, \quad \widehat{T}_{\mathrm{PIM,prefill,i}}(P) = P W_i \tau. \tag{12}$$

For the seven projection categories repeated across 32 blocks, this yields 3.9936, 5.3248, and 10.6496 ms per generated token at the three assumed wave times, respectively. These are *uncalibrated sensitivity inputs*, not published AiM timings or measured DRAMsim3 PIM commands. The model does not charge for input broadcasts, command issue/serialization, inter-bank or cross-PU reductions, or other hardware scheduling. It especially cannot prove efficient execution for FFN-down rows spanning multiple PUs. Decode GEMV occurs after the first token and is therefore *not added* to ordinary prefill TTFT.

Following the paper's separate short-prefill scheduling idea (its Fig. 15), the smaller modeled time is selected for each

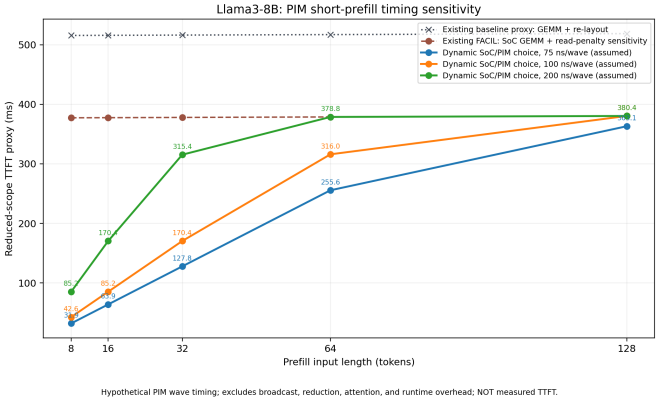


Fig. 7. Optional dynamic SoC/PIM prefill timing sensitivity at hypothetical PIM wave latencies. Values are optimistic component estimates; they exclude PIM staging/reduction overhead and are not observed end-to-end TTFT.

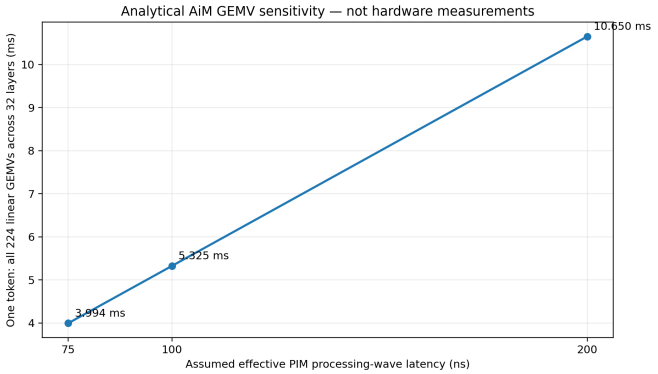


Fig. 8. Analytical total linear-weight PIM GEMV per token versus *assumed* effective wave latency (75/100/200 ns); no PIM hardware measurements are claimed.

prefill projection:

$$\widehat{T}_{\mathrm{dynamic}}(P) = \sum_i 32 n_i \min\left(S_i(P), P W_i \tau\right), \tag{13}$$

$$S_i(P) = T_{\mathrm{SoC,GEMM,i}}(P) + \delta_i. \tag{14}$$

where $\delta_i = 0$ for ideal SoC mapping and $\delta_i = T_{\mathrm{FACIL,read,i}} - T_{\mathrm{conv,read,i}}$ in the sequential-read sensitivity variant. For eight prompt tokens, that sensitivity model produces 31.949, 42.598, and 85.197 ms at 75, 100, and 200 ns/wave, respectively. This is an *additional conditional scenario*, not the measured prefill policy or a replacement for Table VII.

VII. Original Extension: Batch-Aware Decode Dispatch

A. Problem and scope

FACIL's original single-query focus makes PIM-favorable GEMV the natural decode reference, while its already-established short-prefill policy considers SoC/PIM execution selection [1]. Our extension studies a distinct workload: *batched decode*, where B independent sequences supply B token vectors at each decode step. Their linear projections can be evaluated as SoC GEMM; a simple AiM-style PIM model instead executes

B GEMVs. We select *one engine for the entire decode step*, not an independent engine per matrix. This is an analytic policy study, not a claim to invent batched inference or to implement GPU/PIM instruction scheduling.

B. Reproducible decision model

For 32 blocks with 224 FP16 linear projections, we use the same four shapes, multiplicities and peak parameters as the prefill model. For batch width B and hypothetical SoC efficiency η,

$$C_i(B) = \frac{2M_iK_iB}{42.5\times 10^{12}}, \tag{15}$$

$$D_i(B) = \frac{2M_iK_i + 2B(M_i+K_i)}{204.8\times 10^{9}}, \tag{16}$$

$$T_{\mathrm{SoC}}(B,\eta) = \frac{10^3}{\eta}\sum_{i=1}^{4} 32c_i \max(C_i, D_i), \tag{17}$$

where times are in ms, c_i is the per-block projection count, and 42.5 TFLOP/s and 204.8 GB/s are assumed peak quantities, not measured batched GEMM performance. The PIM sensitivity model is

$$T_{\mathrm{PIM}}(B; T_1, \rho) = T_1[1 + (B-1)(1-\rho)], \tag{18}$$

with $T_1 \in \{5.3248, 20\}$ ms. The former uses the *assumed* 100-ns/wave model (53,248 total projection waves); the latter is an *illustrative paper-informed scenario*, *not* a paper-measured linear-layer PIM time. Our primary $\rho = 0$ assumption gives linear batch scaling. Positive ρ is a hypothetical future PIM reuse capability, not established for AiM. The no-transition-cost policy chooses $\min(T_{\mathrm{PIM}}, T_{\mathrm{SoC}})$ at each decode step; the original FACIL hardware does not provide a measured transition cost.

C. Experiments 1–2: crossover and robustness

We sweep $B = 1\ldots 64$, $\eta \in \{0.30, 0.50, 0.763, 1.00\}$ and $\rho \in \{0, 0.25, 0.5\}$. For $\rho = 0$ and $\eta = 0.763$, the first SoC-favorable batches are $B = 17$ (5.3248-ms scenario) and $B = 5$ (20-ms scenario); for $\eta = 0.5$ they are 26 and 7. Across the stated tested parameter combinations, one optimistic low-efficiency, reuse-positive case has no SoC crossover by $B = 64$. These are model breakpoints, not measured GPU/PIM crossover values.

D. Experiments 3–4: changing batches, memory, transitions

We evaluate rising $(1, 4, 16, 32, 64)$, falling and oscillating $(1, 32, 4, 64, 8, 48, 2, 64)$ sequences. Four execution references are kept distinct: (i) always-PIM; (ii) all-SoC; (iii) two simultaneous weight copies, enabling ideal minimum-latency dispatch without re-layout but doubling linear-weight storage from 13.959 to 27.917 GB; and (iv) the paper-style on-demand re-layout strategy, with a full modeled conversion for each SoC pass. The latter is evaluated at 447.284 ms (our serialized DRAM-only full-model stage sum) and a *rough illustrative* 200-ms alternative; the latter is not a controlled replica of the paper's measured re-layout time. FACIL provides one-copy re-layout-free access for the adaptive choice, although runtime transition cost remains unknown. Temporary conversion buffers and metadata memory are excluded from the weight-copy-only comparison.

TABLE VIII
OFFLINE SWITCHING-OVERHEAD TOLERANCE $h_{\max}$ (MS/TRANSITION) UNDER $\eta = 0.763$, $\rho = 0$.

PIM timing	Rising	Falling	Oscillating
100 ns/wave	156.868	156.868	84.037
Illustrative 20 ms/token	78.824	78.824	69.362

TABLE IX
OSCILLATING DECODE TRACE: MODELED LINEAR PROJECTION TOTALS (MS) AT $\eta = 0.763$, $\rho = 0$.

Policy	100 ns/wave	20 ms/vector
Always PIM	1,187.430	4,460.000
Always SoC (best fixed)	722.108	722.108
On-demand (447.284 ms/SoC step)	1,187.430	2,453.426
On-demand (200 ms/SoC step)	1,088.807	1,464.292
Ideal duplication / FACIL, $h = 0$	444.164	593.888
Online FACIL, $h = 10$ ms	514.164	633.351

For unmeasured uniform switch overhead h, an *offline* dynamic program finds the best whole-step SoC/PIM sequence when all future batches are known. Its ideal value is a lower bound on any online dispatch time. We also evaluate a causal hysteresis heuristic which switches when the current step's expected saving exceeds h. Rather than assume a specific h, we report

$$h_{\max} = \max_{\pi: N_\pi > 0,\, T_\pi < T_{\mathrm{static}}} \frac{T_{\mathrm{static}} - T_\pi}{N_\pi}, \tag{19}$$

where T_π is a mixed schedule's zero-overhead total, N_π its transitions and T_{static} the faster of the two fixed-engine totals. This is the largest uniform transition cost below which *some offline schedule* still improves on the best static policy. The first step has a free engine choice, so initialization cost is excluded.

Table IX compares totals for the same eight steps. At $h = 0$, FACIL reduces the best-fixed total by 38.5% and 17.8% in the two scenarios, respectively. Ideal duplication matches that modeled latency and doubles the calculated weight capacity. With an illustrative 10-ms switch charge, the online policy uses seven or three switches, respectively. No hardware switch times enter these results.

VIII. DEVIATIONS FROM THE ORIGINAL FACIL PAPER

- **Mapping integration.** Seo et al. [1] propose system support for page-specific mapping. We use a static 2-MiB page registry, reference Python mapper, and native DRAMsim3 decoder and channel router to isolate address interpretation and placement. OS integration and hardware changes remain outside this prototype.
- **Platform and workload.** The original study combines device evaluation and a modified DRAMsim-based PIM model. We use DRAMsim3 with LPDDR4 commands, LPDDR5-style timing parameters, and synthetic 2-KiB aggregate bank rows. Four FP16 linear projection classes

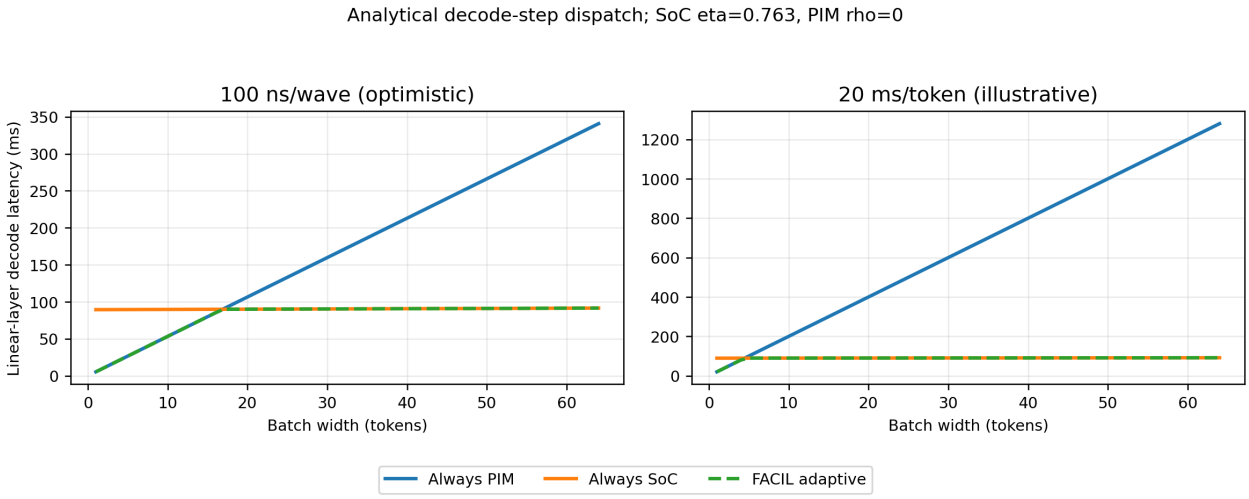


Fig. 9. Linear-projection decode-step latency under two *analytical* PIM timing scenarios ($\eta = 0.763, \rho = 0$). The 0.763 value is an assumed GEMM efficiency sensitivity motivated by the paper's measured GEMV memory utilization; it is *not* measured GEMM efficiency. Attention, KV traffic and real PIM switching are excluded.

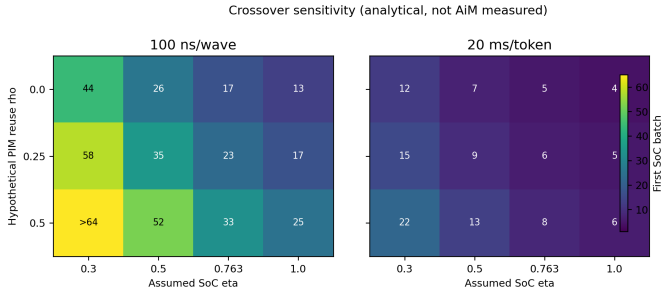


Fig. 10. First SoC-favorable batch versus η and hypothetical PIM reuse ρ. A > 64 cell denotes no crossover within the sweep, not absence of a crossover at every batch size.

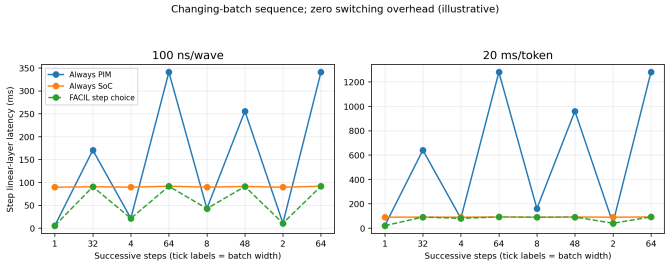


Fig. 11. Oscillating trace's modeled step costs; zero transition cost. Under nonzero overhead, decisions change.

provide a controlled Llama3-8B workload for trace replay and placement checks. Device measurements and calibrated PIM arithmetic would require additional infrastructure.

- **Execution policy.** The original dynamic execution selection addresses prefill. Our main extension chooses one engine per decode step for a batch of independent active sequences. Analytical costs isolate the effects of batch width, assumed reuse, and switch cost. We compare fixed-engine, online, and offline schedules and use the preplaced duplicate layout to expose the storage trade-off.

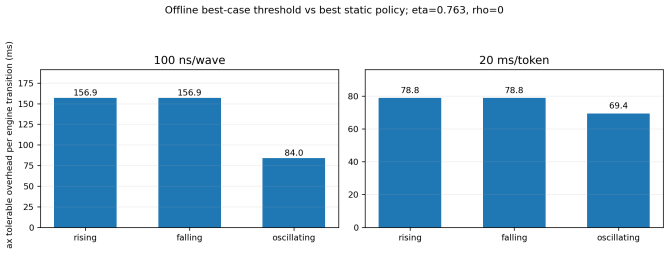


Fig. 12. Maximum tolerable uniform switching cost for an offline mixed-engine schedule to beat the best static choice, under two hypothetical PIM scenarios. Not a measured hardware transition cost.

IX. Discussion and Limitations

A. Modeling limitations and comparison

Timing and protocol fidelity. The study uses DRAMsim3's LPDDR4 command protocol with LPDDR5-style parameters and synthetic 2-KiB aggregate bank rows, not validated physical LPDDR5 or calibrated AiM MAC execution. The original FACIL work uses a different, modified DRAMsim/real-device methodology [1].

Scope of computation. The SoC model charges one weight read per projection at ideal Jetson peak bandwidth/compute, but excludes attention, normalization, nonlinearities, embedding/output layers, kernel launch, caches, and overlapping execution. The baseline copy runs two serialized DRAM stages and omits CPU permutation; it is not a measured end-to-end copy or TTFT.

Mapping penalty extrapolation. Native DRAMsim3 read traces are sequential logical access proxies. Their FACIL-versus-conventional time differences do not directly establish changes

to real GPU GEMM kernels; neither a zero penalty nor the 308.964-ms penalty is a measured SoC GEMM result. Both are presented only as ideal and sensitivity scenarios, avoiding double counting of total reads.

PIM assumptions. All 75/100/200-ns wave costs are hypothetical, with no PIM instruction timing, input broadcast, reduction, or scheduler calibration. The optional PIM short-prefill model is consequently optimistic and illustrative rather than an independently measured speedup.

Reference-paper comparison. FACIL's published Fig. 13 reports 2.89× geometric-mean Jetson TTFT speedup; its Table III reports up to 2.1% Jetson GEMM slowdown on the PIM layout. Its real-world-dataset results (Fig. 15) report 2.37× and 2.63× TTFT speedups after optional dynamic execution selection [1]. The modeled ideal speedups (7.26–7.54×) and extrapolated read-penalty speedups (1.364–1.367×) are *not* apples-to-apples reproductions: they use a restricted linear-only workload, ideal roofline, synthetic DRAM timings, and a different penalty assumption. The no-extra-copy mechanism is reproduced and model trends are studied, not the original percentages.

New decode policy limitations. Batch-width-aware selection is implemented as an analytical step-level policy, not an SoC/PIM runtime. The optimistically modeled 5.325-ms PIM time and illustrative 20-ms time are unmeasured; $\rho > 0$ is hypothetical and $\eta = 0.763$ is borrowed as a *sensitivity setting* from a different GEMV metric. Attention, KV-cache access, GPU kernel launches, time-varying memory contention, real PIM broadcast/reduction, queues and model capacity are excluded. Consequently our modeled decode linear-layer latencies cannot be labeled end-to-end TTLT or hardware speedup. Duplication can match zero-overhead FACIL dispatch latency but requires an additional 13.959 GB of linear weights in the model.

B. Worked numerical example

The geometry gives 2 MiB$/128 = 16$ KiB of page capacity per PU. A K/V row holds $4096 \times 2 = 8192$ bytes, or four 2-KiB chunks. MapID 2 selects four local DRAM rows before the PU changes.

For an 8-MiB K/V matrix, the on-demand re-layout proxy adds 184.560 μs for the source READ and 41.120 μs for the destination WRITE, yielding 225.680 μs of serialized DRAM traffic. At model scale, weighting each matrix class by its multiplicity over 32 blocks gives 447.284 ms. For a 16-token prompt, the modeled SoC linear projection term is 68.567 ms, so the baseline component proxy is 515.851 ms. The FACIL sensitivity case adds 308.964 ms of extrapolated mapped-read difference to the same projection term, giving 377.531 ms and a baseline-to-FACIL ratio of 1.366×. This calculation combines simulated traffic with an analytical projection cost. The preplaced K/V comparison has a different scope because it starts with both address regions already available.

For decode, $\eta = 0.763$, $\rho = 0$, and a 100-ns wave cost give $T_{\mathrm{PIM}}(16) = 16(5.3248) = 85.197$ ms and $T_{\mathrm{SoC}}(16) = 68.567/0.763 \approx 89.865$ ms. At $B = 17$, the PIM cost rises to 90.522 ms, while the SoC cost remains near 90 ms. The cost model therefore changes its preferred engine between these adjacent batch widths.

X. Reproducibility

The experiments use a fixed four-channel, two-rank DRAMsim3 configuration and the FP16 matrix shapes in Table I. Mapping and placement are verified by `facil_mapper.py` and native C++ decoder tests. Trace construction and DRAM traffic are reproducible with `generate_verified_baseline_traces_2k.py`, `measure_relayout.py` and `facil_step3_ffn_reads.py`. The analytical SoC/PIM models use `llama3_soc_gemm_roofline.py` and `facil_pim_gemv_analytical.py`; batch-width sweeps and transition policies use `facil_batch_exp12.py` and `facil_batch_exp34.py`. The preplaced two-copy study retains `run.py`, `verify_decoder.cc`, and the native traces and outputs in `personB/results/facil_two_copy_k_v_2k/`.

The relevant configuration, trace manifests, transaction counts, analytical parameter values and model results are retained to permit replication. The model does not simulate OS modifications, PIM arithmetic or end-to-end inference; these require additional hardware-level evaluation.

XI. Conclusion

The reduced-scope FACIL reproduction confirms MapID placement and native DRAMsim3 routing on four FP16 Llama3 projection classes, and reports an explicitly qualified 447.284-ms serialized baseline copy-stage sum. The preplaced K/V scan shows a memory trade-off: the conventional copy improves sequential SoC read bandwidth by 4.455×, while duplicating the enumerated linear weights raises calculated capacity from 13 to 26 GiB. The prefill first-token models remain theoretical proxies, not device TTFT measurements. Our independently implemented batch-width-aware decode-step selection model identifies SoC/PIM crossovers under two declared PIM timing scenarios and evaluates changing batches, the memory cost of an ideal duplication baseline, and the offline tolerance for unmeasured switching overhead. The analytic model can match ideal two-copy dispatch latency with a single weight copy under FACIL, but proves no real-device speedup.

These results establish an implemented and evaluated student-scale policy extension, with explicit performance and fidelity limits.

XII. Future Work

Measure engine transitions. Implement a timestamped test harness on a SoC–PIM prototype or calibrated execution platform. Measure both directions and compare each transition with an otherwise identical same-engine step. Break down queue drain, synchronization, cache maintenance, activation transfer, and engine startup. Record the distributions under varying batch sizes and load, then replay those costs in the online and offline policies.

Run end-to-end inference. Evaluate complete prefill and decode on representative prompt lengths and arrival patterns. Include attention, KV-cache traffic, normalization, nonlinear activations, sampling, and runtime scheduling. Check numerical agreement against a reference and record actual peak memory and energy alongside latency.

Define success against matched baselines. Compare the extension with always-SoC, always-PIM, and preplaced duplication using the same weights, request traces, and quality criteria. Report median and 95th-percentile time to first token and inter-token latency, throughput, energy per token, peak memory, and measured switch counts and costs. Repeat runs and report uncertainty. Test whether the adaptive policy improves latency or throughput at equal output quality and under a stated memory budget, and identify conditions where the fixed policies win.

REFERENCES

[1] S. H. Seo, J. Kim, D. Lee, S. Yoo, S. Moon, Y. Park, and J. W. Lee, "FACIL: Flexible DRAM Address Mapping for SoC–PIM Cooperative On-device LLM Inference," in *Proc. IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2025, doi:10.1109/HPCA61900.2025.00127.

[2] S. Li, Z. Yang, D. Reddy, A. Srivastava, and B. Jacob, "DRAMsim3: A cycle-accurate, thermal-capable DRAM simulator," *IEEE Computer Architecture Letters*, vol. 19, no. 2, pp. 106–109, 2020, doi:10.1109/LCA.2020.2973991.

[3] University of Maryland Memory Systems Research Group, "DRAMsim3 source repository," https://github.com/umd-memsys/DRAMsim3.